WEEK 3 · LESSON 17 OF 24 · HANDOUT

Understanding and Trusting Image Models

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=NmLK_WQBxB4 — open the lesson online to watch inside the course.

🎓 **Go deeper:** Crash Course AI: Algorithmic Bias and Fairness — https://www.youtube.com/watch?v=gV0_raKR2UQ

🎓 **Go deeper:** MIT 6.S191 (2024): Building AI Models in the Wild — https://www.youtube.com/watch?v=ZAGiinWiFsE

Your goal in this lesson: Evaluate an image model beyond accuracy, see which pixels drive its decisions with a saliency map, and recognise shortcut learning and dataset bias.

Accuracy is not understanding

An image model can score 99% and still be useless — or dangerous — in the field. Famous examples:

- A model that "detected" pneumonia partly by reading **hospital markers** printed on X-rays: one hospital had more sick patients.
- Skin-cancer models that learned that photos with a **ruler** in them were more often cancer (doctors measure suspicious spots).
- Animal classifiers that recognised "wolf" from **snow** in the background.

This is **shortcut learning**: the model finds the easiest pattern in the training data, not the one we meant. Your job is to check.

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

def make_shapes(n, kinds=("circle", "square", "triangle"), size=32, seed=0):
    # n noisy grey images of random shapes at random places and sizes
    rng = np.random.default_rng(seed)
    yy, xx = np.mgrid[0:size, 0:size]
    images = np.zeros((n, size, size), dtype=np.float32)
    labels = rng.integers(0, len(kinds), n)
    for i, lab in enumerate(labels):
        r = rng.uniform(5, 10)
        cx, cy = rng.uniform(r + 1, size - r - 1, 2)
        dx, dy = xx - cx, yy - cy
        kind = kinds[lab]
        if kind == "circle":
            mask = dx ** 2 + dy ** 2 <= r ** 2
        elif kind == "square":
            mask = (abs(dx) <= r * 0.85) & (abs(dy) <= r * 0.85)
        elif kind == "triangle":
            mask = (dy >= -r) & (dy <= r) & (abs(dx) <= (dy + r) / 2)
        elif kind == "ring":
            mask = (dx ** 2 + dy ** 2 <= r ** 2) & (dx ** 2 + dy ** 2 >= (0.55 * r) ** 2)
        elif kind == "diamond":
            mask = abs(dx) + abs(dy) <= r
        else:  # cross
            mask = ((abs(dx) <= 0.3 * r) & (abs(dy) <= r)) | ((abs(dy) <= 0.3 * r) & (abs(dx)
<= r))
        images[i][mask] = rng.uniform(0.6, 1.0)
        images[i] += rng.normal(0, 0.15, (size, size))
    images = torch.tensor(np.clip(images, 0, 1)).unsqueeze(1)   # (n, 1 channel, 32, 32)
    return images, torch.tensor(labels)

X_all, y_all = make_shapes(3000)
X_train, y_train = X_all[:2400], y_all[:2400]
X_test, y_test = X_all[2400:], y_all[2400:]
names = ["circle", "square", "triangle"]

fig, axes = plt.subplots(1, 8, figsize=(12, 2))
for ax, img, lab in zip(axes, X_train, y_train):
    ax.imshow(img[0], cmap="gray")
    ax.set_title(names[lab])
    ax.axis("off")
plt.show()
print(X_train.shape, torch.bincount(y_train))
```

```
class ShapeCNN(nn.Module):
    def __init__(self, n_classes=3):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 32 ->
16
            nn.Conv2d(16, 32, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 16 -> 8
            nn.Flatten())
        self.head = nn.Sequential(nn.Linear(32 * 8 * 8, 64), nn.ReLU(), nn.Linear(64,
n_classes))

    def forward(self, x):
        return self.head(self.features(x))

def train_model(model, X, y, epochs=5, batch=64, lr=1e-3, params=None, augment=None):
    opt = torch.optim.Adam(params if params is not None else model.parameters(), lr=lr)
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(len(X))
        for start in range(0, len(perm), batch):
            idx = perm[start:start + batch]
            xb = X[idx]
            if augment is not None:
                xb = torch.stack([augment(img) for img in xb])
            opt.zero_grad()
            F.cross_entropy(model(xb), y[idx]).backward()
            opt.step()
    return model

def accuracy(model, X, y):
    model.eval()
    with torch.no_grad():
        return round((model(X).argmax(dim=1) == y).float().mean().item(), 3)
```

```
torch.manual_seed(0)
cnn = train_model(ShapeCNN(), X_train, y_train, epochs=5)
print("test accuracy:", accuracy(cnn, X_test, y_test))
```

Step 1 — Where are the mistakes?

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cnn.eval()
with torch.no_grad():
    probs = F.softmax(cnn(X_test), dim=1)
pred = probs.argmax(dim=1)
ConfusionMatrixDisplay(confusion_matrix(y_test, pred), display_labels=names).plot(cmap="Blues")
plt.show()

wrong = torch.where(pred != y_test)[0]
fig, axes = plt.subplots(1, min(6, len(wrong)), figsize=(10, 2))
for ax, i in zip(axes, wrong[:6]):
    ax.imshow(X_test[i, 0], cmap="gray")
    ax.set_title(f"{names[y_test[i]]}→{names[pred[i]]}", fontsize=9)
    ax.axis("off")
plt.show()
```

Most mistakes are small shapes, where a triangle, square and circle look almost alike in 32 × 32 noisy pixels. That is a sensible weakness — a shortcut would look different.

Step 2 — Is the model confident when it is wrong?

```
confidence = probs.max(dim=1).values
print("average confidence when right:", round(confidence[pred == y_test].mean().item(), 3))
print("average confidence when wrong:", round(confidence[pred != y_test].mean().item(), 3))
```

A healthy model is **less confident** on its mistakes. That lets you build a safety rule: if confidence is below, say, 0.8, ask a human expert instead of answering.

Step 3 — A saliency map: which pixels mattered?

The gradient of the class score with respect to the **input pixels** shows which pixels would change the decision most. Bright pixels = important pixels.

```
def saliency(model, image, target):
    model.eval()
    x = image.unsqueeze(0).clone().requires_grad_(True)
    score = model(x)[0, target]
    score.backward()
    return x.grad.abs()[0, 0]

fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for k in range(5):
    img, lab = X_test[k], y_test[k].item()
    axes[0, k].imshow(img[0], cmap="gray"); axes[0, k].set_title(names[lab]); axes[0,
k].axis("off")
    axes[1, k].imshow(saliency(cnn, img, lab), cmap="hot"); axes[1, k].axis("off")
plt.suptitle("Top: image. Bottom: pixels that drive the decision")
plt.show()
```

The bright pixels should sit on the **edges and corners of the shape**, not on the noisy background. If a leaf-disease model's saliency lit up the farmer's hand or the soil, you would have found a shortcut. More advanced tools (Grad-CAM, SHAP) follow the same idea.

Step 4 — Test on data from the real world

A model is only trustworthy on data like its training data. Ask:

- **Who and where?** Were photos taken on the same phones, in the same regions, light and seasons as the real users?
- **Is anyone missing?** Skin-condition models trained mostly on light skin perform worse on dark skin. Crop models trained on one country's varieties can fail in another.
- **Is the test set independent?** Photos of the same plant in both training and test sets inflate the score.

Collect a small **field test set** from the real setting — even 100 photos — before trusting a model.

Robustness check: a simple shift

```
noisy = (X_test + 0.25 * torch.randn_like(X_test)).clamp(0, 1)
print("clean test:", accuracy(cnn, X_test, y_test), " extra noise:", accuracy(cnn, noisy,
y_test))
```

Accuracy drops when the images are noisier than in training. Real cameras and conditions vary — test for it, and augment for it.

Key terms

- **Shortcut learning** — the model uses an easy but wrong clue in the data.
- **Saliency map** — input-gradient picture of which pixels drive a decision.
- **Confidence** — the highest predicted probability.
- **Distribution shift** — real data differs from training data.
- **Field test set** — examples collected from the real place of use.

Remember this: Trust an image model only after checking its mistakes, its confidence, the pixels it uses, and its performance on real, representative data.

Name: _____ Date: _____

Activity

Task 1

Find the confidence threshold below which half of the mistakes would be sent to a human. How many correct answers would also be sent?

Task 2

Show saliency maps for three of the model's mistakes. What did it look at?

Task 3

Make a test set where every shape is small (radius 5–6). How does accuracy change?

Task 4

Write five questions you would ask before deploying a crop-disease app in Rwanda.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is shortcut learning?
   - A. Training faster with a GPU
   - B. Using fewer layers
   - C. The model relies on an easy but wrong clue, like a ruler in the photo
   - D. Skipping validation
2. What does a saliency map show?
   - A. The training loss
   - B. The model size
   - C. Which input pixels most affect the class score
   - D. The labels
3. A healthy model's confidence on its mistakes should be…
   - A. Lower than on correct answers
   - B. Exactly 50%
   - C. Higher than on correct answers
   - D. Always 100%
4. A skin model trained mostly on light skin works worse on dark skin. What is this?
   - A. Dataset bias: a group is under-represented
   - B. Data augmentation
   - C. A bug in PyTorch
   - D. Overfitting to noise
5. What is a field test set?
   - A. Random noise
   - B. Examples collected from the real place of use
   - C. Augmented images
   - D. Training data
6. Why might the same plant appearing in both training and test sets be a problem?
   - A. It breaks PyTorch
   - B. It inflates the test score
   - C. It is not a problem
   - D. It lowers accuracy

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.